

AGIRH — Rapport d'État d'Avancement

Agent RH agentique pour l'Onboarding et l'Offboarding — ENSA Safi, GIDIA

Réalisé par : M. Komla Edem Wilfried Tsetse

Encadrant : M. Moulay Rachid Didi Alaoui (superviseur)

Année universitaire 2025–2026

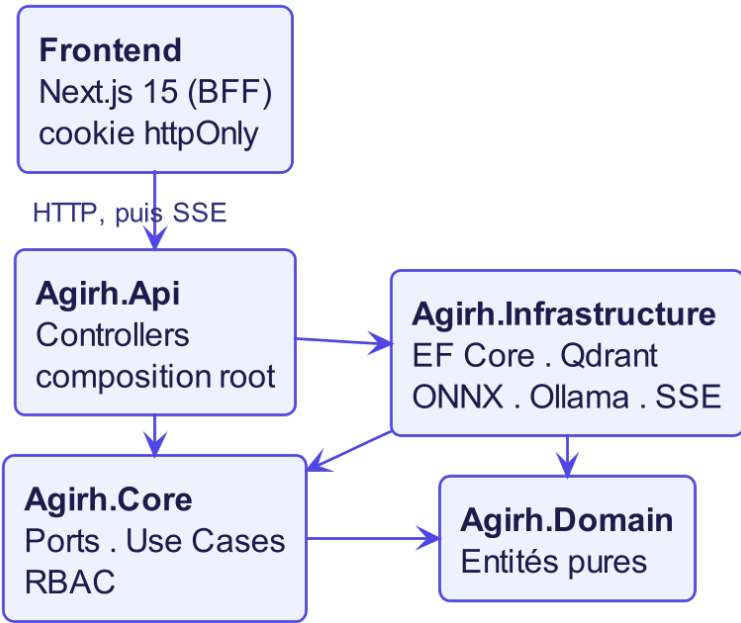
1. Contexte et stratégie

Le stage vise la réalisation d'un agent conversationnel dédié à l'onboarding et à l'offboarding des collaborateurs : répondre aux questions sur les politiques internes (RAG documenté) et informer sur l'avancement d'un dossier, **sans jamais exécuter d'action destructrice ou irréversible** depuis la conversation — une contrainte de conception assumée, pas une limite technique.

Une première itération architecturale a été ré-auditée en cours de stage : pipeline RAG sans reranking ni index vectoriel dédié, boucle d'auto-correction non fonctionnelle en configuration réelle (bug de bouclage confirmé en code), et un périmètre métier qui dérivait au-delà du sujet réel. Une refonte ciblée a été menée pour corriger ces limites et recentrer strictement le périmètre sur l'onboarding/offboarding. **Ce rapport décrit l'état actuel, pleinement fonctionnel de bout en bout.**

2. Architecture et stack technique

Le backend (.NET 8) suit une **architecture hexagonale** à 4 couches (Domain / Core / Infrastructure / Api), règle unique : une couche ne dépend que de celles listées en dessous. Bénéfice concret : remplacer SQL Server, Qdrant ou Ollama ne touche que la couche Infrastructure, et la logique métier se teste sans la moindre infrastructure réelle.



Composant	Choix et usage
Backend	ASP.NET Core, EF Core — ports/use cases/RBAC dans Agirh.Core , adaptateurs dans Agirh.Infrastructure
Frontend	Next.js 15, pattern BFF — le JWT n'est jamais exposé au navigateur, cookie httpOnly posé côté serveur
Auth & RBAC	JWT + ASP.NET Identity, 3 rôles (Employee/HR/QualityAdmin), portée RH vérifiée <i>avant</i> le rôle (DepartmentScopeGuard)
Temps réel	SSE (Server-Sent Events) — un seul mécanisme pour le chat en streaming et les notifications
Données	SQL Server (entités métier) + Qdrant (index vectoriel RAG, ANN/HNSW, cosinus)
Déploiement	Docker Compose (4 services), Ollama natif sur l'hôte (accès GPU/modèles entreprise)

3. Le cœur du projet : le sous-système IA

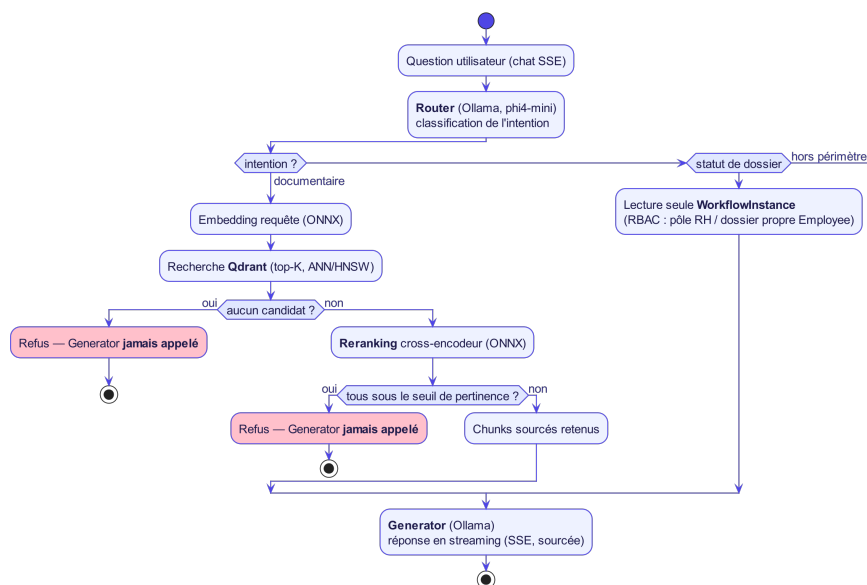
3.1 Pipeline RAG — 4 phases obligatoires

1. **Chunking structurel** (sections Markdown, pas de découpage à taille fixe) : budget de 400 tokens/chunk avec 15% de recouvrement entre chunks voisins, pour ne jamais perdre une information à cheval sur une frontière de découpage.
2. **Embedding** multilingue (corpus en français) en **ONNX Runtime .NET pur**, 768 dimensions — aucun appel Ollama pour cette étape, contrôle total de la latence.
3. **Storage Qdrant**, recherche ANN/HNSW en similarité cosinus — corpus réel indexé et vérifié (6 documents → 72 chunks).
4. **Reranking** cross-encodeur ONNX (**bge-reranker-v2-m3**) — phase obligatoire, pas une amélioration optionnelle : c'est elle qui distingue un vrai pipeline RAG d'un RAG naïf. Un score élevé mesure une proximité thématique, jamais une preuve que la réponse est présente — le seuil de pertinence (0,01) sert de filtre de bruit, pas de garantie à lui seul.

3.2 Orchestration et garde-fou anti-hallucination

Un **Router** (Ollama, **phi4-mini:3.8b**) classe chaque question en trois intentions. Sa sortie n'est **jamais** utilisée telle quelle : validée contre un enum fermé, tout ce qui ne correspond pas exactement retombe sur *hors périmètre* par défaut — **fail-safe, pas fail-open**.

Le point le plus important du projet : **deux portes de sortie anticipée, en code**, avant tout appel au **Generator**. Si Qdrant ne retourne aucun candidat, ou si aucun ne passe le seuil de pertinence après reranking, le Generator n'est jamais invoqué — le système répond directement qu'il n'a pas trouvé l'information. Ce n'est pas une consigne de prompt que le modèle pourrait ignorer : c'est vérifiable et couvert par des tests dédiés.

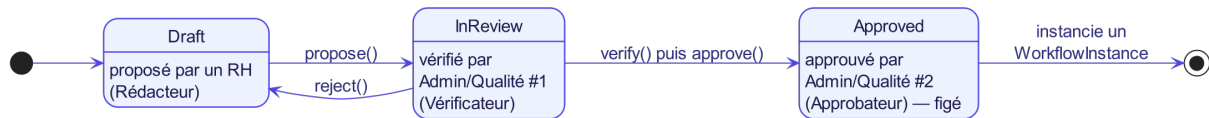


Limite mesurée et assumée : le Router se trompe sur ~27% d'un jeu de 48 questions test (isolé derrière un port, sans impact sur le garde-fou anti-hallucination). Doubler les exemples few-shot n'a eu aucun effet mesuré — limite de raisonnement d'un modèle 3,8B, documentée plutôt que masquée.

4. Workflows métier

Onboarding : le RH du pôle crée une fiche collaborateur (Poste, Pôle, Contrat, Date) → résolution des items attendus contre le *template approuvé* correspondant (croisé Poste × Pôle × Contrat) → **WorkflowInstance** + checklist persistées → items cochés indépendamment (audit trail) → clôture puis archivage en lecture seule définitive.

Circuit qualité des templates : un RH **propose** → un Admin/Qualité **vérifie** → un second **approuve** — seul un template *Approved* peut instancier un dossier réel, contrainte vérifiée par le code, pas seulement documentée. Trois rôles distincts empêchent qu'une seule personne valide unilatéralement son propre travail.



5. Bilan vérifié et suite

211 tests automatisés (xUnit/Moq/FluentAssertions), **0 warning** au build. Pipeline RAG et orchestration vérifiés en HTTP réel de bout en bout (ingestion → question → réponse sourcée, streaming token par token). Déploiement Docker Compose (4 services + Ollama natif) vérifié sur base fraîche : inscription → connexion → chat sourcé, de bout en bout.

Suite prévue : affiner le Router (nouvelle approche à évaluer, le simple ajout d'exemples n'ayant pas suffi) ; endpoints de lecture/liste pour des vues RH au-delà du chat ; trois cas particuliers métier encore ouverts (mutation inter-pôle, annulation/suspension, pôle vacant).